

05

Directives



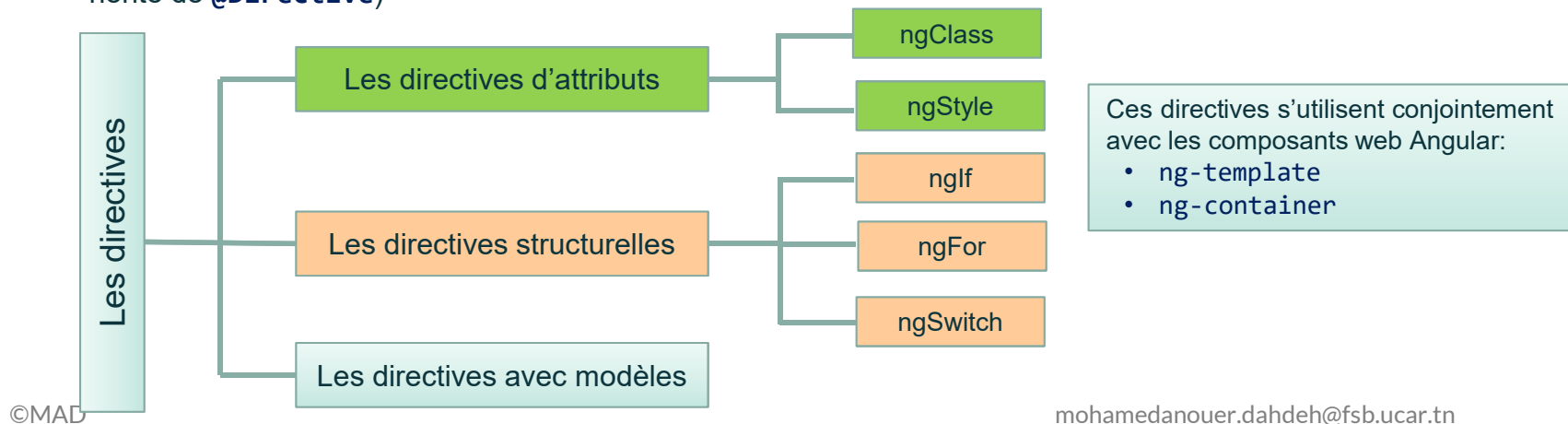
Les directives Angular

- **Les directives** sont des instructions intégrées dans le **DOM** (comme les composants et les sélecteurs des balises HTML)
- **Les directives** sont des éléments clés qui nous permettent non seulement de rendre des vues à l'aide de composants, mais aussi de modifier l'apparence générale et la structure du DOM
- La plupart du temps, **les directives** dans Angular commencent par **ng-** où ng signifie Angular et étend le HTML.
- En TypeScript, **une directive** est une classe à laquelle on applique le décorateur **@Directive**.



Les types de directives

- Il existe trois type de directives :
 - **Les directives structurales** : Elles ont pour but de modifier le DOM en ajoutant, enlevant ou remplaçant un élément du DOM.
 - **Les directives par attribut**: Elles ont pour but de modifier l'apparence ou le comportement d'un élément.
 - **Les directives avec modèle** : Les composants sont des directives avec un template (@Component hérite de @Directive)



ngTemplate

- La directive **ng-template** est utilisée dans Angular pour définir un contenu qui ne sera pas rendu immédiatement, mais qui pourra être utilisé plus tard comme modèle de rendu. Cela peut être utile dans plusieurs situations, telles que :
 - Créer un modèle de rendu qui sera utilisé plusieurs fois dans une vue.
 - Créer un modèle de rendu qui ne sera utilisé que si certaines conditions sont remplies.
- Pour utiliser la directive **ng-template** dans Angular :
 - Dans votre template HTML, définissez un conteneur qui sera utilisé comme modèle de rendu en utilisant la directive **ng-template**.
 - Pour utiliser ce modèle de rendu, vous pouvez utiliser la directive ***ngTemplateOutlet** et lui passer le modèle en utilisant la référence définie précédemment

©MA

```
1 <!--Defining the template-->
2 <ng-template #myTemplate>
3   <p>This is my reusable template</p>
4 </ng-template>
5
6 <!--Using the template -->
7 <div *ngTemplateOutlet="myTemplate"></div>
```

```
1 <!-- You can also pass contexts to your template using the
   ngTemplateOutlet attribute with the context property-->
2 <ng-template #myTemplate let-name="name">
3   <h3> Hello {{name}}! </h3>
4 </ng-template>
5
6 <div *ngTemplateOutlet="myTemplate; context: {name: 'Adam'}">
   </div>
```

*ngFor

- Permet de répéter un traitement sur les éléments d'une collection
- L'élément sur lequel **NgFor** est placé sera répété pour chaque élément de l'itérable.
- S'utilise comme un attribut de balise et sa valeur est une instruction itérative **TypeScript**

```
1 <!-- *ngFor shorthand syntax-->
2 <div *ngFor="let item of items">{{item.name}}</div>
3
4 <!-- With a local variable for the index -->
5 <div *ngFor="let item of items; let i = index">{{i}}:
  {{item.name}}</div>
6
7 <!-- With a local variable to know whether it is an even item -->
8 <div *ngFor="let item of items; let i=index; let isEven = even">
9   {{i}}: {{item.name}} is {{isEven ? 'even': 'odd'}}
10 </div>
```

```
about.component.html X
src > app > about > about.component.html > div.card.border-primary.mb-3 > div.card
19
20 <h3>Messages</h3>
21 <ul>
22   <li *ngFor="let c of comments; let i=index">
23     #{{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}}
24   </li>
25 </ul>
26
```

Messages

- #0 My first comment -- 18:45:07
- #1 My second comment -- 18:45:15
- #2 Another comment -- 18:45:25

*ngFor

- Il est parfois très utile de connaître l'index, par exemple, pour afficher une numérotation à chaque tour de boucle. Il existe donc 5 variables locales :
 - **index**: position de l'item. Commence à 0.
 - **first**: booléen indiquant si l'item est le premier de l'itération
 - **last**: booléen indiquant si l'item est le dernier de l'itération
 - **even**: booléen indiquant si la position de l'item est paire
 - **odd**: booléen indiquant si la position de l'item est impaire

```
about.component.html X
src > app > about > about.component.html > div.card.border-primary.mb-3
20 <h3>Messages</h3>
21 <ul>
22   <li *ngFor="let c of comments; let i=index; let isFirst = first" >
23     #{{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- first : {{isFirst}}
24   </li>
25 </ul>
```

Messages

- #0 My first comment -- 18:55:21 -- first : true
- #1 My second comment -- 18:55:33 -- first : false
- #2 Another comment -- 18:55:53 -- first : false

*ngIf

- Permet d'ajouter/supprimer un élément DOM selon une condition

```
1 <!-- *ngIf : shortland syntax-->
2 <div *ngIf="condition">Content to render when condition is true.</div>
3 <!-- *ngIf : full syntax-->
4 <div *ngIf="condition; else elseBlock">Content to render when condition is true.</div>
5
6 <ng-template #elseBlock>Content to render when condition is false.</ng-template>
```

- Possibilité d'ajouter un traitement **then else**

ng-template: comme son nom l'indique, désigne un *modèle* et peut être utilisé pour imbriquer plusieurs directives structurales ensemble

```
<div *ngIf="comments.length>0 ; then listComments else noComments">
  <!--This code will never be executed !-->
</div>
<ng-template #listComments>
  <!--List comments -->
</ng-template>
<ng-template #noComments>
  <p>No Comments</p>
</ng-template>
```

*ngIf: Exercice

/*TODO*/

Afficher seulement les commentaires avec indice pair

```
<h3>Messages</h3>
<ul>
  <li *ngFor="let c of comments; let i=index; let isFirst = first; let isEven=even"
      *ngIf="isEven">
    {{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- first : {{isFirst}}
  </li>
</ul>
```

Erreur :

ERROR in src/app/about/about.component.html:22:11 - error NG5002: Can't have multiple template bindings on one element. Use only one attribute prefixed with *

```
22      *ngIf="isEven">
      ~~~~~
```

La combinaison directe de **ngFor** et **ngIf** sur le même élément, comme ci-dessus, entraînera une erreur car Angular ne permet pas d'utiliser deux directives structurales sur le même élément.

Solution : utiliser un `<ng-container>` qui est un élément de groupe qui n'affecte pas la mise en page ou le style

```
<ul>
  <ng-container *ngFor="let c of comments; let i=index; let isFirst=first; let isEven=even">
    <li *ngIf="isEven">
      {{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- First : {{isFirst}}
    </li>
  </ng-container>
</ul>
```

©MAD

Messages

- #0 My first comment -- 13:27:38 -- First : true
 - #2 Another comment -- 13:27:49 -- First : false
- mohamedanouer.dahdeh@fsb.ucar.tn

*ngSwitch

- La directive **ngSwitch** permet d'afficher un élément selon la valeur d'une variable,
- Prenons un exemple classique , **la création d'onglets** :
- Le principe est simple :
 - ngSwitch** prend la valeur de la valeur **tab**
 - tab** change selon l'onglet cliqué
 - ngSwitchCase** fait une comparaison avec sa valeur et la valeur de **ngSwitch**. Si l'égalité est positive alors l'élément s'affiche.
 - ngSwitchDefault** affiche l'élément si aucun des **ngSwitchCase** n'est positif.
 - Nous initialisons **tab** à 1 pour être sur le premier onglet lors de l'affichage du composant.

```
TS my-second.component.ts U X
src > app > my-second > TS my-second.component.ts > ...
1 import { Component } from '@angular/core';
2 @Component({
3   selector: 'app-my-second',
4   template: `<ul>
5     <li [ngClass]="{active: tab==1}" (click)="tab=1"><a href="#">Home</a></li>
6     <li [ngClass]="{active: tab==2}" (click)="tab=2"><a href="#">Profile</a></li>
7     <li [ngClass]="{active: tab==3}" (click)="tab=3"><a href="#">Messages</a></li>
8   </ul>
9   <div [ngSwitch]="tab">
10     <div *ngSwitchCase="1">Content 1</div>
11     <div *ngSwitchCase="2">Content 2</div>
12     <div *ngSwitchCase="3">Content 3</div>
13     <div *ngSwitchDefault>Default</div>
14   </div>`,
15   styleUrls: ['./my-second.component.css']
16 })
17 export class MySecondComponent {
18   tab: number = 1;
19 }
```

- [Home](#)
- [Profile](#)
- [Messages](#)

Content 1

ngStyle

- Modifie le style d'un élément HTML
- S'utilise conjointement avec le **property binding** pour récupérer des valeurs définies



```
1 <!-- ngStyle : syntax-->
2 <div [ngStyle]="{style:expression}"></div>
```

```
export class AboutComponent implements OnInit {
  colors = {odd:"red", even:"green"}
```

*/*TODO*/*
Afficher les commentaires
d'indice pair en **vert** et
ceux d'indice impair en
rouge

```
<div *ngIf="comments.length>0 ; else noComments">
  <h3>Messages</h3>
  <ul>
    <li *ngFor="let c of comments; let i=index; let isFirst = first; let isEven=even let isOdd=odd">
      <ng-container *ngIf="isEven">
        <span [ngStyle]="{color:colors.even}"> #{{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- first : {{isFirst}}</span>
      </ng-container>
      <ng-container *ngIf="isOdd">
        <span [ngStyle]="{color:colors.odd}"> #{{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- first : {{isFirst}}</span>
      </ng-container>
    </li>
  </ul>
</div>
```



Messages

- #0 My first comment -- 21:18:24 -- first : true
- #1 My second comment -- 21:18:37 -- first : false
- #2 Other comment -- 21:18:44 -- first : false

ngClass

- Permet d'attribuer de nouvelles classes à un élément HTML
- S'utilise conjointement avec le **property binding** pour récupérer des valeurs

```
1 <!-- ngStyle : syntax-->
2 <div [ngClass]="{'class1': expression1, 'class2': expression2, ... }"></div>
```

*/*TODO*/*

Afficher les commentaires
d'indice pair en **vert** et
background light et ceux
d'indice impair en **rouge**

```
about.component.html X
src > app > about > about.component.html > div.card.border-primary.mb-3 > div.card-body
18 <h3>Messages</h3>
19 <ul>
20 <li *ngFor="let c of comments; let i=index; let isFirst = first; let isEven=even let isOdd=odd">
21 <span [ngClass]="{'text-success bg-light' : isEven, 'impair': isOdd}">
22   {{i}} {{c.message}} -- {{c.date | date:'HH:mm:ss'}} -- first : {{isFirst}}
23 </span>
24 </li>
25 </ul>
26 </div>
27
28
```

Classes Bootstrap

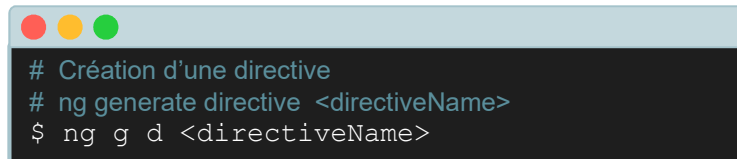
```
# about.component.css X
src > app > about > # about.component.css > ...
1 .impair{ color : red; }
```

Messages

- #0 My first comment -- 09:07:42 -- first : true
- #1 My second comment -- 09:07:51 -- first : false
- #2 Other comment -- 09:08:03 -- first : false

Comment créer une directive personnalisée ?

- On peut définir nos propres directives pour attacher un comportement personnalisé aux éléments du DOM.
- En TypeScript, une **directive** est une classe à laquelle on applique le décorateur **@Directive**.



```
# Création d'une directive
# ng generate directive <directiveName>
$ ng g d <directiveName>
```

Custom directive

- Supposons que vous souhaitiez créer une directive qui change la couleur de fond d'un élément lorsqu'on passe la souris dessus.

```
TS highlight.directive.ts U X 2
src > app > directives > TS highlight.directive.ts > ...
1 import { Directive, ElementRef, HostListener } from '@angular/core';
2
3 @Directive({
4   selector: '[appHighlight]'
5 })
6 export class HighlightDirective {
7   constructor(private el: ElementRef) { }
8   @HostListener('mouseenter') onMouseEnter() {
9     this.highlight('yellow');
10  }
11   @HostListener('mouseleave') onMouseLeave() {
12     this.highlight('');
13   }
14   private highlight(color: string) {
15     this.el.nativeElement.style.backgroundColor = color;
16   }
17 }
```

©MAD

```
# Création d'une directive 1
$ ng g d directives/highlight
```

```
<> my-first.component.html U X 3
src > app > my-first > <> my-first.component.html > ...
8
9 <p appHighlight>
10   | Hover over me to see the color change!
11 </p>
```

Hover over me to see the color change!

Info

Cette directive utilise les écouteurs d'événements (`@HostListener`) pour détecter lorsque la souris entre dans et quitte l'élément auquel la directive est appliquée. Lorsque la souris entre, la méthode `onMouseEnter()` est appelée et applique un style de fond jaune à l'élément. Lorsque la souris quitte, la méthode `onMouseLeave()` est appelée et réinitialise le style de fond à sa valeur par défaut.

Exercice : Directive

#01 BNF

Voir les dates des commentaires d'indice pair en **vert** et ceux d'indice impair en **rouge**

Messages

My First comment	09:55:36
My second comment	09:55:43
Other comment	09:55:56

Directive

#01 BNF

Voir les dates des commentaires d'indice pair en **vert** et ceux d'indice impair en **rouge**

```
17 <div *ngIf="comments.length>0 ; else noComments">
18   <h3>Messages</h3>
19   <ul class="list-group">
20     <li class="list-group-item d-flex justify-content-between align-items-center"
21       *ngFor="let c of comments">
22       {{c.message}}<span class="badge badge-secondary badge-pill">{{c.date | date:'HH:mm:ss'}}</span>
23     </li>
24   </ul>
25 </div>
```

about.component.html X

src > app > about > about.component.html > div.card.border-primary.mb-3 > div.card-body

```
17 <div *ngIf="comments.length>0 ; else noComments">
18   <h3>Messages</h3>
19   <ul class="list-group">
20     <li class="list-group-item d-flex justify-content-between align-items-center"
21       *ngFor="let c of comments; let isOdd=odd; let isEven=even">
22       {{c.message}}<span class="badge badge-pill" [ngClass]='{"badge-success": isEven, "badge-danger":isOdd }'>
23       {{c.date | date:'HH:mm:ss'}}</span>
24     </li>
25   </ul>
26 </div>
```

Messages

My First comment

09:55:36

My second comment

09:55:43

Other comment

09:55:56

@fsb.ucar.tn

06

Pipes



Pipe

▪ Définition :

- Un **pipe** a le rôle de formater et modifier l'affichage d'une donnée dans la vue du **composant**
- Un pipe est une classe décorée par le décorateur **@Pipe**
- Implémente la méthode **transform(value: unknown, ...args: unknown[])** de l'interface **PipeTransform**
- Angular fournit plus d'une douzaine de pipes intégrés pour couvrir les cas d'utilisation courants.



```
<!-- Pipe syntax -->
<!-- The syntax of Angular pipes is inspired by that of Unix shell.-->
{{ "hello" | uppercase }} <!-- HELLO -->
<!--Some pipes can take parameters:-->
{{ myDate | date:'d MMM y' }} <!-- 18 Feb 2024 -->
<!--It is possible to chain pipes: -->
{{ myDate | date:'d MMM y' | uppercase }} <!-- 18 FEB 2024 -->
```

Créer un Pipe personnalisé

```
# Création d'un pipe
$ ng generate pipe pipes/get-char
```

```
TS get-char.pipe.ts U X
src > app > pipes > TS get-char.pipe.ts > ...
1  import { Pipe, PipeTransform } from '@angular/core';
2
3  @Pipe({
4    name: 'getChar'
5  })
6  export class GetCharPipe implements PipeTransform {
7    transform(value: unknown, ...args: unknown[]): unknown {
8      return null
9    }
10 }
```

*/*TODO*/*

Créer un **pipe** qui permet de récupérer un caractère d'une chaîne de caractères à une position donnée. si la position n'est pas correcte, retourner le premier caractère.

```
transform(value: unknown, position: unknown): unknown {
  if (typeof position === 'number' && typeof value === 'string') {
    if (position && position < value.length)
      return value.charAt(position);
    return value.charAt(0);
  }
  else return "";
}
```

```
<p> {{ "hello" | getChar:2 }} </p> <!-- print: l -->
```

Exercice : Pipe

#02 BNF

Voir la date d'un commentaire
au format :

'2020 NOV FRIDAY 20 – 23:25:26'

Messages

My first comment	2020 NOV FRIDAY 20-11:14:01
My second comment	2020 NOV FRIDAY 20-11:14:21
Other comment	2020 NOV FRIDAY 20-11:14:29

Documentation : <https://angular.io/api/common/DatePipe>

Solution: Pipe

#02 BNF

Voir la date d'un commentaire
au format :

'2020 NOV FRIDAY 20 - 23:25:26'

Messages

My first comment	2020 NOV FRIDAY 20-11:14:01
My second comment	2020 NOV FRIDAY 20-11:14:21
Other comment	2020 NOV FRIDAY 20-11:14:29

```
about.component.html X
src > app > about > about.component.html > div.card.border-primary.mb-3 > div.card-body
17 <div *ngIf="comments.length>0 ; else noComments">
18 <h3>Messages</h3>
19 <ul class="list-group">
20 <li class="list-group-item d-flex justify-content-between align-items-center">
21   *ngFor="let c of comments; let isOdd=odd; let isEven=even">
22     {{c.message}}<span class="badge badge-pill" [ngClass]='{"badge-success": isEven, "badge-danger":isOdd }'>
23       {{c.date | date:'HH:mm:ss'}}</span>
24   </li>
25 </ul>
26 </div>
```

```
about.component.html X
src > app > about > about.component.html > div.card.border-primary.mb-3 > div.card-body
17 <div *ngIf="comments.length>0 ; else noComments">
18 <h3>Messages</h3>
19 <ul class="list-group">
20 <li class="list-group-item d-flex justify-content-between align-items-center">
21   *ngFor="let c of comments; let isOdd=odd; let isEven=even">
22     {{c.message}}<span class="badge badge-pill" [ngClass]='{"badge-success": isEven, "badge-danger":isOdd }'>
23       {{c.date | date:'y MMM EEEE d - HH:mm:ss' | uppercase}}</span>
24   </li>
25 </ul>
26 </div>
```

07

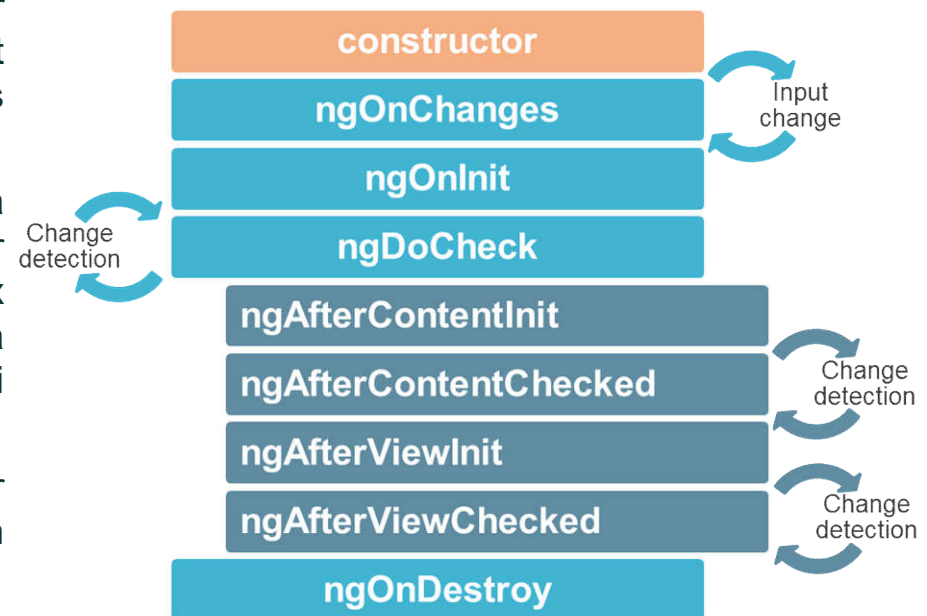
Interaction entre les composants



LifeCycle d'un composant

- Une instance de composant a un cycle de vie qui commence lorsque Angular instancie la classe du composant et présente la vue du composant avec ses vues enfants.
- Le cycle de vie se poursuit avec la détection des modifications, car Angular vérifie quand les propriétés liées aux données changent et met à jour à la fois la vue et l'instance de composant si nécessaire.
- Le cycle de vie se termine lorsque Angular détruit l'instance du composant et retire son template du DOM.
- Angular fournit des **méthodes de hook** pour exploiter les événements clés du cycle de vie d'un composant.

©MAD



mohamedanouer.dahdeh@fsb.ucar.tn

Angular Hook

- **ngOnChanges:** appelée après le constructeur et chaque fois que les valeurs input changent. La méthode reçoit un objet SimpleChanges qui contient les valeurs actuelles et précédentes des propriétés annotées d'@Input().
- **ngOnInit:** appelée une seule fois. C'est là que l'initialisation du composant doit avoir lieu, tel que la récupération des données initiales. En effet, les composants doivent être peu coûteux à construire, les opérations coûteuses doivent donc être tenues à l'écart du constructeur. Le constructeur ne doit pas faire plus que donner des valeurs initiales simples aux variables de la classe.
- **ngDoCheck:** appelée immédiatement après ngOnChanges à chaque exécution du cycle de détection du changement, et immédiatement après ngOnInit lors de la première exécution. Donne la possibilité de mettre en œuvre un algorithme de détection du changement personnalisé.

Angular Hook

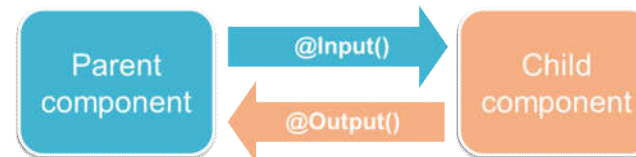
- **ngAfterContentInit:** appelée une seule fois. Invoquée après qu'Angular ait effectué une projection de contenu dans la vue du composant
- **ngAfterContentChecked:** appelée après `ngAfterContentInit` et chaque `ngDoCheck` suivant.
- **ngAfterViewInit:** appelée une seule fois. Appelée lorsque la vue du composant a été complètement initialisée.
- **ngAfterViewChecked:** appelée après `ngAfterViewInit` et chaque `ngDoCheck` suivant.

Info

Pour chaque hook du cycle de vie il existe une interface correspondante. Leurs noms sont dérivés du nom du hook de cycle de vie correspondant sans le **ng**. Par exemple, pour utiliser **ngOnInit()**, implémentez l'interface **OnInit**.

Formes d'interaction

- Une application Angular est constituée de plusieurs composants
- **1^{ère} forme d'interaction**
 - En utilisant **des formulaires** et des **liens**, on peut envoyer des données d'un composant à un autre
- **Autres formes d'interaction : parent-enfant**
 - Ajouter le sélecteur d'un 1^{er} composant dans le Template d'un 2^{ème} composant
 - on appelle le 1^{er} composant : composant fils
 - on appelle le 2^{ème} composant : composant parent
 - Définir le sens de transmission de données par
 - un composant web Angular : `ng-content`
 - des décorateurs `@Input()`, `@Output()`, `@ViewChild`, `@ViewChildren`, `@ContentChild`, `@ContentChildren`



Interaction : ng-content (Projection du contenu)

```
export class ParentComponent implements OnInit {  
  constructor() { }  
  ngOnInit() {}  
}
```

```
export class ChildComponent implements OnInit {  
  constructor() { }  
  ngOnInit() {}  
}
```

```
<p>parent works!</p>  
<app-child>Value 1 </app-child>  
<app-child>Value 2 </app-child>  
<app-child>Value 3 </app-child>
```

```
<p>child works! :  
  <ng-content></ng-content>  
</p>
```

- Tout ce qui est écrit entre les balises du **composant enfant** dans le **composant parent** est injecté dans le Template de l'enfant et remplace les balises **<ng-content>**

parent works!

child works! : Value 1

child works! : Value 2

child works! : Value 3

Interaction : @Input()

```
export class ParentComponent implements OnInit {  
  parentProprety : any = 'valueOfParentProprety';  
  constructor() { }  
  ngOnInit() {}  
}
```

```
export class ChildComponent implements OnInit {  
  @Input() childProprety : any;  
  constructor() { }  
  ngOnInit() {}  
}
```

```
<p>parent works!</p>  
<app-child [childProprety] = "parentProprety"></app-child>
```

```
<p>child works!</p>  
<p> {{ childProprety }} </p>
```

- L'ajout du décorateur `@Input()` sur une propriété d'un composant enfant signifie qu'il peut recevoir sa valeur de son composant parent. Le composant parent transmet cette valeur via property binding dans son template.

parent works!

child works!

valueOfParentProprety

mohamedanouer.dahdeh@tsb.ucar.tn

Interaction : @ViewChild()

- **Objectif**

- Récupérer les données du 1^{er} composant fils à partir d'un composant parent

- **Comment ?**

- Déclarer un composant fils **comme attribut** d'un composant parent et le décorer avec **@ViewChild()**
- Utiliser cet attribut pour récupérer les données souhaitées

Interaction : @ViewChild()

```
src > app > parent > TS parent.component.ts > ParentComponent
4 @Component({
5   selector: 'app-parent',
6   templateUrl: './parent.component.html',
7   styleUrls: ['./parent.component.css']
8 })
9
10
11 export class ParentComponent implements OnInit {
12
13   @ViewChild(ChildComponent, {static:false}) child : ChildComponent;
14
15   parentPropety : any = 'valueOfParentPropety';
16
17   constructor() { }
18
19   ngOnInit() {
20     console.log(this.child.childPropety);
21   }
22 }
```

```
src > app > child > TS child.component.ts > ChildComponent
9
10 export class ChildComponent implements OnInit {
11
12   @Input() childPropety : any ;
13
14   constructor() { }
15
16   ngOnInit() { }
17 }
18
19
```

```
parent.component.html
src > app > parent > parent.component.html > ...
1
2
3 <p>parent works!</p>
4 <app-child
5   [childPropety] = "parentPropety">
6 </app-child>
```

```
child.component.html
src > app > child > child.component.html > p
1
2
3 <p>child works!</p>
4 <p> {{ childPropety }} </p>
5
6
7
```

```
✖ ERROR TypeError: Cannot read property 'childPropety' of undefined
at ParentComponent.ngOnInit (parent.component.ts:20)
at checkAndUpdateDirectiveInline (core.js:31910)
```

- `{ static: false }` car les attributs du composant enfant sont alimentés par le père et ne sont donc pas initialisés dans le composant.

Interaction : @ViewChild()

```

src > app > parent > TS parent.component.ts > ParentComponent
7   styleUrls: ['./parent.component.css']
8 })
9
10 export class ParentComponent implements OnInit {
11
12   @ViewChild(ChildComponent, {static:true}) child : ChildComponent;
13
14   parentPropety : any = 'valueOfParentPropety';
15
16   constructor() { }
17
18   ngOnInit() {
19     console.log(this.child.childPropety);
20   }
21 }

```

```

child > TS child.component.ts > ChildComponent > ChildComponent
9
10 export class ChildComponent implements OnInit {
11
12   @Input() childPropety : any = "valueChild";
13
14   constructor() { }
15
16   ngOnInit() {}
17
18 }
19

```

```

child > child.component.html > child.component.html > ...
src > app > child > child.component.html > ...
1 <p>child works!</p>
2 <p> {{ childPropety }} </p>
3
4
5

```

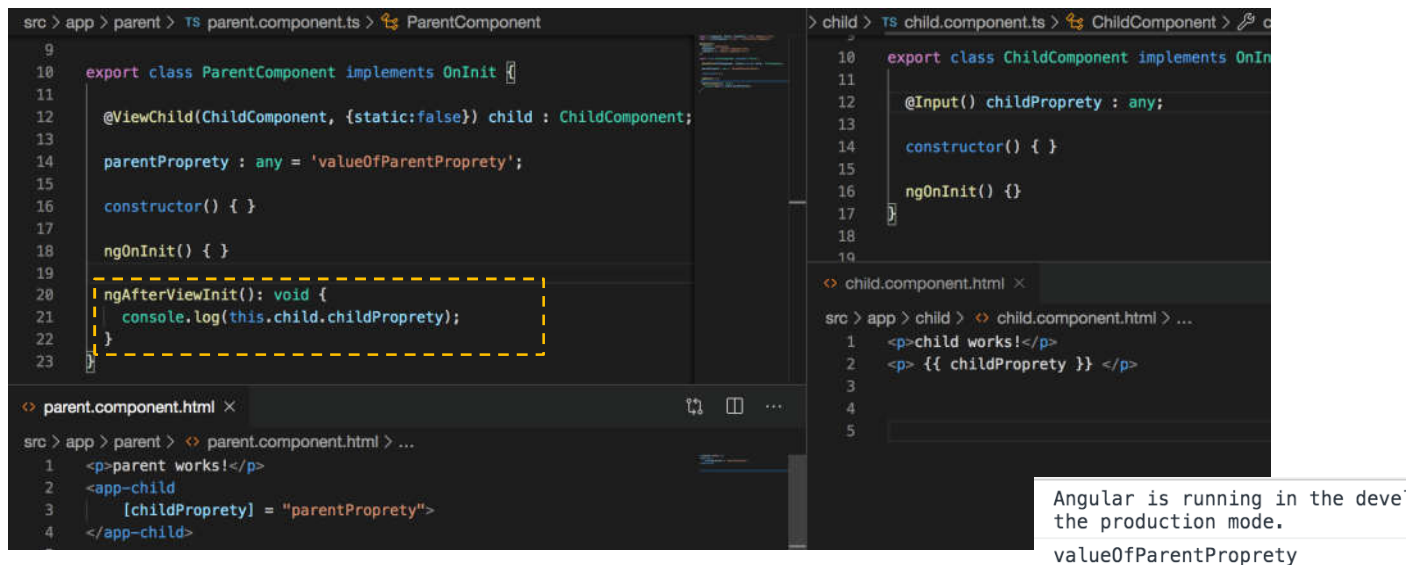
```

src > app > parent > parent.component.html > ...
1 <p>parent works!</p>
2 <app-child
3   [childPropety] = "parentPropety">
4 </app-child>
5

```

- En mettant `{ static: true }`, on indique qu'il ne faut plus attendre l'initiation de la vue (fils) car ses données sont statiques

Interaction : @ViewChild()



```
src > app > parent > TS parent.component.ts > ParentComponent
9
10 export class ParentComponent implements OnInit {
11
12   @ViewChild(ChildComponent, {static:false}) child : ChildComponent;
13
14   parentPropety : any = 'valueOfParentPropety';
15
16   constructor() { }
17
18   ngOnInit() { }
19
20   ngAfterViewInit(): void {
21     console.log(this.child.childPropety);
22   }
23
24 }
25
```

```
> child > TS child.component.ts > ChildComponent > ChildComponent
10 export class ChildComponent implements OnInit {
11
12   @Input() childPropety : any;
13
14   constructor() { }
15
16   ngOnInit() { }
17
18 }
19
```

```
<> parent.component.html <
src > app > parent > <> parent.component.html > ...
1 <p>parent works!</p>
2 <app-child
3   [childPropety] = "parentPropety">
4 </app-child>
```

```
<> child.component.html <
src > app > child > <> child.component.html > ...
1 <p>child works!</p>
2 <p> {{ childPropety }} </p>
3
4
5
```

Angular is running in the development mode.
valueOfParentPropety

- Remettre { static: false }, pour récupérer la valeur du père afficher la propriété dans ngAfterViewInit()

Interaction : @Output()

- Les composants enfants communiquent avec leurs parents à l'aide d'événements.
- L'ajout du décorateur `@Output()` sur une propriété de type `EventEmitter` d'un composant enfant permet aux données de circuler de l'enfant vers le parent. Le composant parent peut réagir à l'événement via la syntaxe d'event binding.

```
TS parent.component.ts x
src > app > parent > TS parent.component.ts > ParentComponent > children
10 export class ParentComponent implements OnInit {
11
12   moyenne: number = 0;
13   somme: number = 0;
14   nbr: number = 0;
15   children = ['A', 'B', 'C'];
16
17   constructor() { }
18
19   ngOnInit() { }
20   computeAvg(note: number) {
21     this.somme += note;
22     this.nbr++;
23     this.moyenne = this.somme / this.nbr;
24   }
25 }

TS child.component.ts x
src > app > child > TS child.component.ts > ChildComponent >
10 export class ChildComponent implements OnInit {
11
12   @Input() nom: string;
13   @Output() message = new EventEmitter<number>();
14   note: number;
15   buttonStatus: boolean = false;
16
17   constructor() { }
18
19   ngOnInit() { }
20
21   send() {
22     this.message.emit(this.note);
23     this.buttonStatus = true;
24   }
25 }

parent.component.html x
src > app > parent > parent.component.html > ...
1 <h3> Moyenne de mes enfants {{ moyenne }} </h3>
2 <app-child *ngFor="let child of children" [nom]="child"
3   (message)="computeAvg($event)">
4 </app-child>
5

child.component.html x
src > app > child > child.component.html > ...
1 <h6> {{ nom }} </h6>
2 <input type="number" name="note" [(ngModel)]="note">
3 <button (click)="send()" [disabled]="buttonStatus">
4   Send
5 </button>
```

Moyenne de mes enfants 3.33

A
1 Send

B
4 Send

C
5 Send

Exercice : Interaction entre les composants

*/*TODO*/*

Créer un composant pour afficher les commentaires.
Ce composant sera appelé par AboutComponent

Solution : Interaction entre les composants

```
# Création d'un nouveau composant 1
$ ng generate component comment
```

```
TS comment.component.ts U X 2
1 import { Component, Input, OnInit } from '@angular/core';
2 import { Comment } from '../shared/comment';
3
4 @Component({
5   selector: 'app-comment',
6   templateUrl: './comment.component.html',
7   styleUrls: ['./comment.component.css']
8 })
9 export class CommentComponent implements OnInit{
10   @Input() commentList:Comment[];
11   constructor(){}
12   ngOnInit(): void {
13   }
14 }
```

```
<> comments.component.html X 3
src > app > comments > <> comments.component.html > ...
1 <div *ngIf="commentList.length>0 ; else noComments">
2   <h3>Messages</h3>
3   <ul class="list-group">
4     <li class="list-group-item d-flex justify-content-between align-items-center"
5       *ngFor="let c of commentList; let isOdd=odd; let isEven=even">
6       {{c.message}}<span class="badge badge-pill" [ngClass]="{'badge-success': isEven, 'badge-danger':isOdd}">
7       {{c.date | date:'y MMM EEEE d - HH:mm:ss' | uppercase}}</span>
8     </li>
9   </ul>
10 </div>
11 <ng-template #noComments>
12   <p>No Comments</p>
13 </ng-template>
```

```
<> about.component.html U X 4
21 <app-comment [commentList]="comments"></app-comment>
```

08

Atelier 2

